I'm currently developing a linux diagnostics and monitoring tool and want to understand what all I should implement in the tool, i.e., system like what os, kernel, cpu, how many cores? temperature, gpu name and temp, etc? network, etc. Please provide me with all that such a tool should monitor/expose such that it is professional and is useful for the devs/community, etc

If you want your Linux diagnostics/monitoring tool to feel **professional and genuinely useful to developers, sysadmins, and power users**, it should cover several major categories of system information and metrics. Most mature tools (like htop, glances, netdata, btop, and neofetch) combine subsets of the following.

Ask ChatGPT

I'll structure this like a **feature specification** so you can design your tool architecture around modules.

---

1. System Information (Static / Rarely Changing)

These help identify the machine and environment.

OS & Kernel

- Distribution name
- Distribution version
- Kernel version
- Kernel build

- Kernel architecture
- Boot parameters
- Hostname
- Uptime
- Boot time
- Init system (systemd, openrc, etc)

Example:

```
OS: Ubuntu 24.04
Kernel: 6.8.0-25-generic
Arch: x86_64
Init: systemd
Uptime: 2 days 5 hours
```

---

2. CPU Information

Important for performance debugging.

Static CPU Info

- CPU model name
- Vendor (Intel/AMD)
- Microarchitecture
- Number of sockets
- Cores per socket
- Threads per core
- Total logical CPUs

- Cache sizes (L1/L2/L3)
- CPU flags (SSE, AVX, etc)
- Base clock
- Max boost clock
- Virtualization support

Live CPU Metrics

- CPU usage per core
- Total CPU usage
- CPU frequency per core
- CPU temperature
- Load average (1,5,15)
- Context switches/sec
- Interrupt rate
- CPU throttling state
- CPU steal time (important for VMs)

Sources:

```
/proc/cpuinfo
/sys/devices/system/cpu
/proc/stat
```

---

3. GPU Monitoring

Important for developers using compute or graphics workloads.

GPU Info

- GPU name
- Vendor
- Driver version
- VRAM total
- GPU architecture

Live GPU Metrics

- GPU utilization %
- VRAM usage
- GPU temperature
- GPU power usage
- GPU clock
- Fan speed
- Encoder/decoder usage

Tools to integrate with:

- nvidia-smi (NVIDIA)
- radeontop (AMD)
- intel_gpu_top (Intel)

---

4. Memory Monitoring

RAM

- Total RAM

- Used RAM
- Free RAM
- Cached memory
- Buffers
- Available memory
- Memory pressure

Swap

- Swap total
- Swap used
- Swap in/out rate

Advanced

- HugePages
- Memory fragmentation
- OOM events
- Page faults

Sources:

```
/proc/meminfo
/proc/vmstat
```

---

5. Storage / Disk

Disk Info

- Disk model
- Disk type (HDD/SSD/NVMe)
- Capacity
- Sector size
- Mount points
- Filesystem type

Disk Usage

- Used / free space
- Disk utilization %
- Inodes used/free

Disk Performance

- Read speed
- Write speed
- IOPS
- Queue length
- Latency

SMART health

- Disk temperature
- Reallocated sectors
- Wear level (SSD)

Tools:

- smartctl

Sources:

```
/proc/diskstats
/sys/block
```

---

6. Filesystem Monitoring

For each mount:

- Mount point
- Filesystem type
- Capacity
- Used
- Free
- Inodes used
- Read/write throughput
- Mount options

Example:

```
/dev/nvme0n1p2 -> /
ext4
80% used
```

---

7. Process Monitoring

This is a **must-have for professional tools**.

Process Table

For each process:

- PID
- User
- CPU %
- Memory %
- Thread count
- Priority / nice
- State
- Runtime
- Command

Advanced

- Per-process IO
- Open file descriptors
- Memory maps
- CPU affinity
- Cgroup

Sources:

```
/proc/[pid]
```

8. Network Monitoring

Network Interfaces

- Interface name
- MAC address
- IP addresses
- MTU
- Link speed
- Driver

Traffic

- RX bytes/sec
- TX bytes/sec
- Packets/sec
- Errors
- Drops

Connections

- Active TCP connections
- UDP sockets
- Listening ports

Network stats

- TCP retransmits
- SYN backlog
- packet drops

Sources:

```
/proc/net/dev
/proc/net/tcp
```

---

9. Temperature & Sensors

Hardware health is very useful.

Monitor:

- CPU temperature
- GPU temperature
- Motherboard temperature
- VRM temperature
- Disk temperature
- Fan speeds
- Voltages

Tool integration:

- lm-sensors

Sources:

```
/sys/class/hwmon
```

---

10. Power Monitoring (Laptop / Servers)

If supported:

- Battery level
- Battery health
- Power consumption
- AC adapter status
- Charging rate

Sources:

```
/sys/class/power_supply
```

---

11. Container & Virtualization Detection

Important for modern environments.

Detect:

- Running inside Docker
- Running inside Kubernetes
- VM hypervisor

Hypervisor examples:

- KVM
- VMware
- VirtualBox

Metrics:

- cgroup resource limits
- container CPU usage
- container memory usage

---

12. System Load / Performance Metrics

Useful for diagnosing slow systems.

Monitor:

- Load average
- Run queue length
- CPU steal time
- IO wait
- Scheduler stats
- context switches
- interrupts/sec

---

13. Security Monitoring (Advanced)

Some advanced diagnostics tools include:

- Logged-in users
- Last login
- SSH sessions

- Failed login attempts
- SELinux status
- AppArmor status
- Firewall status

---

14. Logs

Expose system logs:

- Kernel logs
- System logs
- Service logs

Via:

- journalctl

Features:

- log tail
- error highlighting
- filter by service

---

15. Hardware Inventory

Useful for bug reports.

Detect:

Motherboard

- Vendor
- Model
- BIOS version

PCI devices

- GPUs
- Network cards
- Storage controllers

Tool integration:

- lspci

USB devices

- Vendor
- Product
- Device class

Tool:

- lsusb

---

16. Performance Profiling (Advanced)

For developers:

- CPU flame graphs
- syscall tracing

- perf events

Tools to integrate:

- perf
- strace
- bpftrace

---

17. Alerts / Thresholds

Professional tools allow:

Examples:

```
CPU > 90%
RAM < 500MB
Disk > 95%
Temperature > 85°C
```

Then:

- CLI alerts
- logs
- notifications

---

18. Export / Integration

Critical for devs and ops.

Support:

- JSON output
- Prometheus metrics
- CSV export
- REST API

Monitoring ecosystem:

- Prometheus
- Grafana

---

19. Historical Metrics

Optional but powerful.

Store:

- CPU usage history
- network usage history
- disk IO history

Time windows:

```
1m
5m
1h
24h
```

---

20. Nice UI (Very Important)

Choose interface type:

Terminal UI

Like:

- btop
- htop

Features:

- graphs
- colors
- sorting
- filtering

Web dashboard

Like:

- netdata

Bonus: Unique Features That Would Make Your Tool Stand Out

Ideas many tools don't combine well:

1️⃣ One-command system report

```
sysdiag --report
```

Outputs a **full machine report for bug reports**.

2️⃣ Developer debugging mode

Show:

- syscalls/sec
- scheduler latency
- page faults
- kernel pressure

3️⃣ Bottleneck detector

Automatically detect:

```
CPU bottleneck
memory pressure
disk IO bottleneck
network congestion
```

4️⃣ Remote monitoring agent

Run on servers and expose:

```
HTTP API
SSH query
```

```
Prometheus exporter
```

Recommended Modular Architecture

Structure the project like:

```
core/
  collector/
    cpu
    memory
    disk
    network
    gpu
    sensors

  exporters/
    cli
    json
    prometheus
    web

  ui/
    tui
    web

  alerts/
    thresholds
```

If you want, I can also show you:

- **The complete `/proc` and `/sys` map for Linux monitoring tools** (this is extremely useful for building one).
- **A full architecture design for a professional open-source monitoring tool**.
- **Features used by top tools like `btop`, `glances`, and `netdata`.**

↪ * **The complete `/proc` and `/sys` map for Linux monitoring tools** (this is extremely useful for building one). * **A full architecture design for a professional open-source monitoring tool**.

yes please; also if you could provide the code for the headings 1-10

Below is a **practical developer-oriented guide** for building a Linux diagnostics/monitoring tool. I'll include:

1. **Important `/proc` and `/sys` files you should read**
2. **A professional architecture design**
3. **Example code for headings 1–10** (implemented in **Python** because it's concise and easy to prototype; these translate easily to Rust/C/Go)

Many mature tools like htop, btop, glances, and netdata rely heavily on these same kernel interfaces.

---

1. The Complete `/proc` and `/sys` Map for Monitoring Tools

Linux exposes most runtime metrics via **pseudo-filesystems**.

Think of it like:

```
kernel -> exposes metrics -> /proc and /sys -> monitoring tool reads them
```

`/proc` (runtime kernel statistics)

CPU

```
/proc/stat
/proc/cpuinfo
/proc/loadavg
/proc/uptime
```

Metrics available:

```
CPU usage
per-core stats
interrupts
context switches
boot time
load average
```

ChatGPT Log in Sign up for free

```
/proc/meminfo
/proc/vmstat
/proc/zoneinfo
```

Metrics:

```
total memory
available memory
swap
page faults
OOM events
memory pressure
```

Processes

```
/proc/[pid]/
```

Important files:

```
/proc/[pid]/stat
/proc/[pid]/status
/proc/[pid]/cmdline
/proc/[pid]/io
/proc/[pid]/fd/
```

Metrics:

```
CPU usage
memory usage
threads
file descriptors
IO
```

Disk / IO

```
/proc/diskstats
/proc/mounts
/proc/partitions
```

Metrics:

```
IO reads
IO writes
sectors read
sectors written
IO queue time
```

Network

```
/proc/net/dev
/proc/net/tcp
/proc/net/udp
/proc/net/snmp
/proc/net/netstat
```

Metrics:

```
bandwidth usage
packet drops
```

```
tcp connections
network errors
```

`/sys` (hardware + kernel device model)

`/sys` exposes **structured device information**.

CPU

```
/sys/devices/system/cpu/
```

Important files:

```
cpu*/cpufreq/scaling_cur_freq
cpu*/online
cpu*/topology/core_id
```

Metrics:

```
CPU frequency
core topology
power states
```

Temperature / sensors

```
/sys/class/hwmon/
```

Example:

```
/sys/class/hwmon/hwmon0/temp1_input
/sys/class/hwmon/hwmon0/fan1_input
```

Metrics:

```
CPU temperature
fan speed
voltage
```

Storage

```
/sys/block/
```

Example:

```
/sys/block/nvme0n1/stat
/sys/block/sda/stat
```

Metrics:

```
disk IO
queue length
```

GPU (varies by driver)

AMD:

```
/sys/class/drm/
```

Intel:

```
/sys/class/drm/card0/
```

NVIDIA typically uses:

```
nvidia-smi
```

Power

```
/sys/class/power_supply/
```

Metrics:

```
battery %
charging state
power consumption
```

---

2. Architecture for a Professional Monitoring Tool

A **clean modular design** is critical.

High-level architecture

```
          +--------------------+
          |       CLI/TUI      |
          +---------+----------+
                    |
               presentation
                    |
         +----------+-----------+
         |      core engine     |
         +----------+-----------+
                    |
               metrics manager
                    |
     +-------+-------+-------+-------+
     |       |       |       |       |
    CPU     MEM     DISK    NET     GPU
 collector collector collector collector
```

Suggested project structure

```
sysdiag/

core/
   collector/
       cpu.py
       memory.py
       disk.py
       network.py
```

```
        gpu.py
        sensors.py

    system/
        osinfo.py
        uptime.py
        hardware.py

    process/
        process_monitor.py

ui/
    cli.py
    tui.py

exporters/
    json_exporter.py
    prometheus_exporter.py

alerts/
    threshold_engine.py
```

Data model (recommended)

All collectors return **structured metrics**

Example:

```
{
  "cpu": {
      "usage": 32,
      "cores": 8,
```

```
      "load_avg": [1.1, 0.9, 0.7]
  }
}
```

This allows:

```
CLI
Web dashboard
API
Prometheus exporter
```

to reuse the same data.

---

3. Code Examples for Headings 1–10

I'll provide **minimal but functional examples**.

---

1. System Information

Python Run

```python
import platform
import os

def system_info():
    return {
        "hostname": platform.node(),
        "os": platform.system(),
        "os_version": platform.version(),
```

```
        "kernel": platform.release(),
        "architecture": platform.machine(),
        "uptime": open("/proc/uptime").read().split()[0]
    }

print(system_info())
```

---

2. CPU Information

Python Run

```python
import psutil

def cpu_info():
    return {
        "physical_cores": psutil.cpu_count(logical=False),
        "logical_cores": psutil.cpu_count(),
        "cpu_percent": psutil.cpu_percent(interval=1),
        "per_core": psutil.cpu_percent(interval=1, percpu=True),
        "load_avg": os.getloadavg()
    }
```

---

3. GPU Monitoring (NVIDIA example)

Python Run

```python
import subprocess
import json
```

```python
def gpu_info():
    try:
        result = subprocess.check_output([
            "nvidia-smi",
            "--query-gpu=name,temperature.gpu,utilization.gpu,memory.used,memory.total",
            "--format=csv,noheader,nounits"
        ]).decode()

        gpus = []

        for line in result.strip().split("\n"):
            name,temp,util,mem_used,mem_total = line.split(", ")
            gpus.append({
                "name": name,
                "temp": temp,
                "utilization": util,
                "mem_used": mem_used,
                "mem_total": mem_total
            })

        return gpus
    except:
        return []
```

---

4. Memory Monitoring

Python Run

```python
import psutil

def memory_info():
```

```
    mem = psutil.virtual_memory()
    swap = psutil.swap_memory()

    return {
        "total": mem.total,
        "used": mem.used,
        "available": mem.available,
        "percent": mem.percent,
        "swap_total": swap.total,
        "swap_used": swap.used
    }
```

---

5. Disk Monitoring

Python Run

```python
import psutil

def disk_info():
    disks = []

    for part in psutil.disk_partitions():
        usage = psutil.disk_usage(part.mountpoint)

        disks.append({
            "device": part.device,
            "mount": part.mountpoint,
            "fstype": part.fstype,
            "total": usage.total,
            "used": usage.used,
            "percent": usage.percent
        })
```

```
    return disks
```

---

6. Filesystem Monitoring

Python Run

```python
import shutil

def filesystem_usage(path="/"):
    total, used, free = shutil.disk_usage(path)

    return {
        "path": path,
        "total": total,
        "used": used,
        "free": free
    }
```

---

7. Process Monitoring

Python Run

```python
import psutil

def processes(limit=10):
    plist = []

    for p in psutil.process_iter(['pid','name','cpu_percent','memory_percent']):
```

```
        plist.append(p.info)

    plist.sort(key=lambda x: x['cpu_percent'], reverse=True)

    return plist[:limit]
```

---

8. Network Monitoring

Python Run

```python
import psutil

def network_stats():
    net = psutil.net_io_counters(pernic=True)

    stats = {}

    for iface, data in net.items():
        stats[iface] = {
            "bytes_sent": data.bytes_sent,
            "bytes_recv": data.bytes_recv,
            "packets_sent": data.packets_sent,
            "packets_recv": data.packets_recv
        }

    return stats
```

---

9. Temperature Sensors

Python Run

```python
import psutil

def temperatures():
    temps = psutil.sensors_temperatures()

    result = {}

    for name, entries in temps.items():
        result[name] = []

        for entry in entries:
            result[name].append({
                "label": entry.label,
                "temp": entry.current
            })

    return result
```

---

10. Power / Battery

Python Run

```python
import psutil

def battery():
    batt = psutil.sensors_battery()

    if batt is None:
        return None
```

```
    return {
        "percent": batt.percent,
        "plugged": batt.power_plugged,
        "time_left": batt.secsleft
    }
```

Extra Recommendations (Important for Open Source Success)

Add:

JSON output

```
sysdiag --json
```

machine report

```
sysdiag --report
```

Prometheus endpoint

```
http://localhost:9100/metrics
```

Compatible with:

- Prometheus
- Grafana

If you'd like, I can also show:

- 20 extremely useful `/proc` metrics most monitoring tools forget
- How to build a `btop` -style terminal UI
- How to make the tool insanely fast using Rust instead of Python
- How to build a Netdata-like real-time dashboard